
type: Note cssclasses: - cornell-notes

Back to [[Pembelajaran Mesin]]

[!cornell] Fundamentals of Artificial Neural Networks (ANN) Architecture

Questions/Cues

- Mengapa ANN cocok untuk data atribut kompleks?
- Bagaimana arsitektur FFNN berbeda dengan perceptron?
- Apa fungsi lapisan tersembunyi dalam ANN?
- Mengapa XOR menjadi benchmark penting untuk ANN?
- Bagaimana representasi grafis ANN menjelaskan aliran informasi?

Reference Points

- IF3270 Pembelajaran Mesin (Slides 1-41)
- Mitchell, Machine Learning (1997)
- Raschka, Machine Learning with Pytorch and Scikit-Learn (Chapter 11)
- Goodfellow, Deep Learning (2016)

Pengantar Jaringan Saraf Tiruan

Jaringan Saraf Tiruan (JST/ANN) merupakan model komputasi yang terinspirasi oleh sistem saraf biologis. ANN terdiri dari unit-unit pemrosesan sederhana (neuron) yang saling terhubung dan bekerja secara paralel untuk menyelesaikan tugas pembelajaran. **Keunggulan utama ANN** terletak pada kemampuannya mempelajari pola kompleks dari data mentah dengan banyak atribut, menangani noise dalam data, dan menghasilkan output fleksibel (diskrit, kontinu, atau vektor). ANN cocok untuk masalah dimana: - Data memiliki banyak atribut (high-dimensional) - Output yang diinginkan bisa beragam bentuk - Data training mungkin mengandung kesalahan - Waktu pelatihan yang lama dapat ditoleransi - Interpretasi model tidak menjadi prioritas utama

Arsitektur Dasar ANN

Tiga komponen utama arsitektur ANN: 1. **Lapisan Input:** Menerima data masukan (misal: fitur-fitur vektor) 2. **Lapisan Tersembunyi:** Melakukan

transformasi non-linear melalui fungsi aktivasi 3. **Lapisan Output:** Menghasilkan prediksi akhir **Perceptron** merupakan bentuk ANN paling sederhana dengan hanya lapisan input dan output. **Multi-Layer Perceptron (MLP)** memperkenalkan lapisan tersembunyi yang memungkinkan pemodelan hubungan non-linear kompleks. Setiap neuron pada lapisan tersembunyi menerima input dari seluruh neuron di lapisan sebelumnya (fully connected) dan mengaplikasikan fungsi aktivasi seperti sigmoid atau ReLU. **Contoh analogi:** Bayangkan ANN sebagai pabrik pengolahan data dimana setiap departemen (lapisan tersembunyi) melakukan spesialisasi pengolahan tertentu sebelum hasil akhir dikirim ke manajemen (lapisan output).

Klasifikasi Arsitektur ANN

Feedforward Neural Network (FFNN): - Aliran informasi satu arah (input → hidden → output) - Tidak memiliki siklus/loop (acyclic) - Contoh: MLP untuk klasifikasi gambar **Recurrent Neural Network (RNN):** - Memiliki feedback loop yang memungkinkan "memori" - Cocok untuk data sekuensial (teks, time-series) - **Tidak dibahas detail dalam catatan ini Visualisasi arsitektur** dapat menggunakan representasi grafis dimana node merepresentasikan neuron dan edge merepresentasikan bobot koneksi. Kedalaman (depth) jaringan ditentukan oleh jumlah lapisan tersembunyi.

Penyelesaian Masalah XOR dengan ANN

Masalah XOR menjadi benchmark penting karena: - Tidak dapat diselesaikan oleh perceptron tunggal (linear inseparability) - Membutuhkan minimal satu lapisan tersembunyi dengan dua neuron **Implementasi dengan fungsi aktivasi sigmoid:** - Neuron tersembunyi pertama: $h1 = \sigma(-10 + 20x1 + 20x2)$ - Neuron tersembunyi kedua: $h2 = \sigma(30 - 20x1 - 20x2)$ - Neuron output: $y = \sigma(-30 + 20h1 + 20h2)$ **Implementasi alternatif dengan ReLU:** - Menggunakan fungsi aktivasi ReLU di lapisan tersembunyi - Output linear menunjukkan fleksibilitas desain arsitektur Contoh ini menunjukkan bagaimana lapisan tersembunyi **membuat transformasi non-linear** yang mengubah masalah non-linear menjadi separable secara linear di ruang fitur baru.

[!cornell] ##### Summary

Jaringan Saraf Tiruan (ANN) adalah model komputasi terinspirasi biologis yang terdiri dari neuron-neuron terhubung dalam lapisan input, tersembunyi, dan output. **Feedforward Neural Network (FFNN)** merupakan arsitektur dasar dengan aliran informasi searah tanpa loop, cocok untuk masalah klasifikasi dan regresi kompleks. **Lapisan tersembunyi** memungkinkan ANN mempelajari representasi non-linear, seperti terbukti dalam penyelesaian masalah XOR yang membutuhkan minimal satu

lapisan tersembunyi. **Pemilihan arsitektur** (jumlah lapisan, neuron, fungsi aktivasi) merupakan faktor krusial yang menentukan kemampuan pemodelan ANN.

[!ad-libitum]- Additional Information

Perkembangan Terkini Arsitektur ANN

Dalam dekade terakhir, berkembang berbagai varian arsitektur seperti Convolutional Neural Networks (CNN) untuk pemrosesan gambar, Transformer untuk pemrosesan bahasa alami, dan Graph Neural Networks untuk data berbasis graf. Inovasi terbaru fokus pada arsitektur yang lebih efisien secara komputasi seperti MobileNet untuk aplikasi edge computing dan Neural Architecture Search (NAS) untuk desain otomatis arsitektur optimal.

Tantangan Desain Arsitektur

- **Vanishing/Exploding Gradients:** Masalah pada jaringan sangat dalam yang menyulitkan pelatihan
- **Overfitting:** Risiko tinggi ketika jumlah parameter sangat besar dibanding ukuran dataset
- **Interpretabilitas:** Kesulitan memahami keputusan model kompleks ("black box problem")
- **Resource Intensity:** Kebutuhan komputasi tinggi untuk arsitektur berskala besar

Aplikasi ANN di Berbagai Bidang

1. **Kesehatan:** Deteksi kanker dari citra medis
2. **Keuangan:** Prediksi risiko kredit
3. **Otomotif:** Sistem navigasi otonom
4. **Pertanian:** Prediksi hasil panen berdasarkan data iklim
5. **Energi:** Optimasi distribusi listrik smart grid

Tools untuk Eksperimen

- **TensorFlow Playground:** <https://playground.tensorflow.org>
- **NN-SVG:** Tools visualisasi arsitektur jaringan
- **Keras Tuner:** Otomasi hyperparameter tuning

Bacaan Lanjutan

- "Neural Networks and Deep Learning" oleh Michael Nielsen (online book)
- "Deep Learning Specialization" oleh Andrew Ng (Coursera)

- Jurnal Nature: "Deep learning for predicting complex traits in plants"

type: Note cssclasses: - cornell-notes

Back to [[Pembelajaran Mesin]]

[!cornell] Forward Propagation Mechanisms in FFNN

Questions/Cues

- Bagaimana alur informasi dalam forward propagation?
- Peran fungsi aktivasi dalam perhitungan output?
- Contoh implementasi XOR dengan aktivasi berbeda?
- Perbedaan input vector vs mini-batch processing?
- Bagaimana menghitung parameter di Keras?

Reference Points

- Lecture_ANN_IF3270.pdf (Halaman 12-20, 22-25)
- Goodfellow et al. Deep Learning (Halaman 17-21)
- Raschka Machine Learning with PyTorch (Halaman 13-14)

Mekanisme Dasar Forward Propagation

Forward propagation adalah proses perhitungan output jaringan saraf dengan mengalirkan informasi dari input layer melalui hidden layer menuju output layer. Pada FFNN (Feed Forward Neural Network), aliran ini bersifat satu arah tanpa adanya loop atau siklus. Setiap neuron menerima input berupa bobot (weights) yang dikalikan dengan nilai input sebelumnya, kemudian dijumlahkan dan ditambahkan bias. Hasil penjumlahan ini kemudian diproses oleh fungsi aktivasi untuk menghasilkan output neuron tersebut. Misalnya, untuk neuron hidden layer pertama: $h1 = f(w1 * x1 + w2 * x2 + b)$, dimana f adalah fungsi aktivasi. Contoh analogi: Bayangkan sistem pipa air dimana input adalah tekanan awal, setiap sambungan pipa (neuron) mengatur aliran dengan katup (bobot), dan fungsi aktivasi seperti filter yang mengubah karakteristik aliran.

Fungsi Aktivasi dan Perannya

Fungsi aktivasi menentukan apakah suatu neuron akan "menyala" berdasarkan input yang diterima. Dua fungsi umum: 1. **Sigmoid**: Mengkompres nilai ke rentang 0-1, cocok untuk klasifikasi biner $\sigma(z) = 1/(1+e^{(-z)})$ 2. **ReLU**: Mengembalikan 0 untuk input negatif dan nilai linear untuk positif $\text{ReLU}(z) = \max(0, z)$ Pada contoh XOR, fungsi sigmoid memungkinkan jaringan mempelajari keputusan non-linear. ReLU sering digunakan di hidden layer karena efisiensi komputasi dan mitigasi masalah vanishing gradient.

Studi Kasus: Implementasi XOR

Model Sigmoid (Halaman 15-16): - Hidden layer: $h1 = \sigma(-10 + 20x1 + 20x2)$ $h2 = \sigma(30 - 20x1 - 20x2)$ - Output: $y = \sigma(-30 + 20h1 + 20h2)$ **Model ReLU + Linear** (Halaman 18): - Hidden layer: $h1 = \text{ReLU}(0 + x1 + x2)$ $h2 = \text{ReLU}(-1 + x1 + x2)$ - Output linear: $y = 0 + h1 - 2h2$ Kedua model menunjukkan bagaimana kombinasi fungsi aktivasi dan arsitektur layer dapat menyelesaikan masalah non-linear seperti XOR.

Pemrosesan Input Vector vs Mini-Batch

Input Vector (Single Instance): - Perhitungan dilakukan per sampel - Formula: $f(x) = f2(wy^T * f1(wxh^T * x + c) + b)$ **Mini-Batch** (Multiple Instances): - Diolah secara paralel menggunakan operasi matriks - Formula: $f(X) = f2(f1(X * wxh + c) * why + b)$ - Memanfaatkan optimasi GPU untuk komputasi cepat Contoh XOR mini-batch (Halaman 20) menunjukkan efisiensi komputasi saat memproses banyak data sekaligus.

Implementasi Praktis dengan Keras

Kode dasar FFNN 3-layer (Halaman 24):

```
python model = Sequential()
model.add(Dense(12, input_dim=8, activation='relu')) # Hidden layer 1
model.add(Dense(8, activation='relu')) # Hidden layer 2
model.add(Dense(1, activation='sigmoid')) # Output layer
```

Perhitungan Parameter (Halaman 25): - Layer 1: (8 input + 1 bias) * 12 neuron = 108 - Layer 2: (12 input + 1 bias) * 8 neuron = 104 - Output: (8 input + 1 bias) * 1 neuron = 9 - Total: 221 parameter

[!cornell] ##### Summary **Forward propagation** adalah inti dari proses inferensi FFNN yang menghitung output berdasarkan input dan bobot jaringan. **Fungsi aktivasi** seperti Sigmoid dan ReLU memungkinkan pembelajaran pola non-linear, diimplementasikan dalam contoh XOR dengan dua pendekatan berbeda. Pemrosesan **mini-batch** meningkatkan efisiensi komputasi melalui paralelisasi matriks. Implementasi praktis di Keras menunjukkan bagaimana menghitung parameter dan

membangun arsitektur bertingkat. **Kemampuan FFNN** dalam memodelkan hubungan kompleks bergantung pada konfigurasi bobot, fungsi aktivasi, dan kedalaman jaringan.

[!ad-libitum]- Additional Information

Analisis Kompleksitas Komputasi

Kompleksitas forward propagation untuk jaringan dengan L layer adalah $O(\sum_{l=1}^L n_l * n_{l-1})$, dimana n_l adalah jumlah neuron di layer ke- l . Operasi dominan adalah perkalian matriks-vektor. Pada GPU modern, operasi ini dioptimasi melalui paralelisasi tinggi.

Optimasi Numerik

Teknik seperti batch normalization dan inisialisasi bobot cerdas (He/Glorot) meningkatkan stabilitas numerik selama forward propagation. Untuk jaringan sangat dalam, floating point 16-bit (FP16) sering digunakan untuk mempercepat komputasi.

Edge Cases Penting

1. **Input Nol:** Beberapa fungsi aktivasi seperti ReLU menghasilkan output nol untuk input negatif
2. **Saturasi Fungsi:** Sigmoid dan tanh dapat mengalami saturasi pada nilai ekstrim, menghilangkan gradien
3. **Ketergantungan Bobot:** Inisialisasi bobot tidak tepat menyebabkan semua neuron mempelajari pola sama (symmetry breaking problem)

Proyek Eksperimen Mandiri

1. Bangun FFNN dari scratch menggunakan NumPy untuk dataset MNIST, bandingkan waktu komputasi untuk input single vs batch
2. Eksplorasi pengaruh fungsi aktivasi berbeda (Swish, GELU) pada akurasi model XOR
3. Visualisasi distribusi aktivasi layer menggunakan TensorBoard

Alat dan Referensi Lanjut

- **Tools:** TensorFlow Playground, Netron (Visualizer Arsitektur)
- **Buku:** "Neural Networks and Deep Learning" oleh Michael Nielsen
- **Paper:** "Understanding the difficulty of training deep feedforward neural networks" (Glorot & Bengio, 2010)

- **Dataset:** UCI Machine Repository untuk masalah klasifikasi non-linear

type: Note cssclasses: - cornell-notes

Back to [[IF3270 Pembelajaran Mesin]]

[!cornell] Backpropagation Algorithm and Gradient Computation

Questions/Cues

- Mengapa backpropagation diperlukan di ANN?
- Bagaimana menghitung gradien untuk unit output?
- Bagaimana menghitung gradien untuk unit tersembunyi?
- Apa fungsi learning rate pada update bobot?
- Bagaimana hubungan forward-backward propagation?

Reference Points

- Lecture_ANN_IF3270.pptx (Slides 26-41)
- Machine Learning by Mitchell (1997)
- Deep Learning by Goodfellow et al. (2016)

Konsep Dasar Backpropagation

Backpropagation adalah algoritma pembelajaran untuk jaringan saraf tiruan yang bertujuan meminimalkan error dengan menyesuaikan bobot secara iteratif. Algoritma ini terdiri dari dua fase utama: **forward propagation** (menghitung output) dan **backward propagation** (menghitung gradien error). Analogi: Bayangkan sedang melatih tim pemain bola. Forward propagation seperti pertandingan uji coba untuk melihat performa, sementara backpropagation adalah analisis video pertandingan untuk mengidentifikasi kesalahan dan memperbaiki strategi latihan (bobot jaringan).

Mekanisme Perhitungan Gradien

Untuk unit output: Gradien error (δ) dihitung sebagai selisih antara output aktual dan target, dikalikan turunan fungsi aktivasi: $\delta_{\text{output}} = (y_{\text{pred}} - y_{\text{true}}) * f'(\text{net_output})$ **Untuk unit tersembunyi:** Gradien merupakan akumulasi error

dari unit downstream yang terhubung, dikalikan turunan fungsi aktivasi:
 $\delta_{\text{hidden}} = \sum(\delta_{\text{next}} * w_{\text{next}}) * f'(\text{net_hidden})$ Contoh: Pada jaringan dengan 1 hidden layer, error dari unit output akan "dipropagasikan" ke unit tersembunyi menggunakan bobot antara keduanya.

Prosedur Pembelajaran

1. **Inisialisasi bobot** secara acak
2. **Forward pass:** Hitung output semua layer
3. **Hitung error** di output layer
4. **Backward pass:** Hitung δ untuk tiap layer
5. **Update bobot** dengan aturan: $w_{\text{baru}} = w_{\text{lama}} - \alpha * (\delta * \text{output_layer_sebelumnya})$ α (learning rate) mengontrol besar update bobot. Nilai α terlalu besar menyebabkan overshooting, terlalu kecil memperlambat konvergensi.

Implementasi Algoritma

Backpropagation menggunakan **gradient descent** untuk menurunkan error surface. Proses berulang sampai memenuhi kriteria berhenti: - Jumlah iterasi maksimum tercapai - Error training di bawah threshold - Error validation tidak membaik Contoh nyata: Pengenalan digit tulisan tangan menggunakan arsitektur dengan 784 input (28x28 piksel), hidden layer, dan 10 output (digit 0-9). Setiap digit diklasifikasikan dengan menghitung probabilitas di output layer.

[!cornell] ##### Summary **Backpropagation** adalah algoritma fundamental pelatihan ANN yang meminimalkan error melalui penyesuaian bobot berbasis gradien. Prosesnya melibatkan **dua fase**: forward propagation untuk komputasi output dan backward propagation untuk distribusi error. **Gradien error** dihitung secara rekursif mulai dari output layer ke hidden layer menggunakan aturan rantai. Efektivitas algoritma bergantung pada **learning rate** yang tepat dan kriteria penghentian yang terdefinisi dengan baik.

[!ad-libitum]- Additional Information

Analisis Kompleksitas

Kompleksitas waktu backpropagation adalah $O(n * m * \text{epochs})$ dimana n adalah jumlah sampel training, m jumlah bobot. Kompleksitas ruang $O(w)$ untuk menyimpan bobot dan gradien. Pada jaringan dalam, terjadi **vanishing gradient** ketika gradien menjadi sangat kecil di layer awal, menghambat pembelajaran.

Teknik Optimasi Lanjut

1. **Momentum:** Menambahkan komponen kecepatan untuk menghindari osilasi

$$\Delta w(t) = \alpha * \delta * x + \beta * \Delta w(t-1)$$
2. **Adaptive Learning Rate:** Adam optimizer menyesuaikan learning rate per parameter
3. **Regularisasi L2:** Menambahkan penalty term untuk mencegah overfitting

Batasan Aplikasi

- Tidak menjamin konvergensi ke minimum global
- Sensitif terhadap inisialisasi bobot awal
- Membutuhkan komputasi intensif untuk dataset besar

Proyek Eksplorasi Mandiri

1. Implementasikan backpropagation dari scratch dengan Python menggunakan NumPy
2. Bandingkan konvergensi dengan berbagai optimizer (SGD, Adam, RMSprop)
3. Visualisasi perubahan bobot selama training menggunakan TensorBoard

Bacaan Lanjutan

- "Neural Networks and Deep Learning" oleh Michael Nielsen (Chapter 2)
- "Pattern Recognition and Machine Learning" oleh Bishop (Section 5.3)
- Paper: "Efficient BackProp" oleh Yann LeCun et al.
- Tutorial interaktif: <https://google.github.io/deep-learning-scaling/>

type: Note cssclasses: - cornell-notes

Back to [[Pembelajaran Mesin]]

[!cornell] Backpropagation Practice Exercises and Applications

Questions/Cues

- Bagaimana menghitung error term δ untuk unit output?
- Prosedur update bobot menggunakan learning rate α

- Implementasi backpropagation pada jaringan sederhana
- Perbedaan perhitungan gradien output vs hidden layer
- Strategi terminasi algoritma backpropagation

Reference Points

- Lecture_IF3270_ANN.pptx (Slides 26-41)
- Machine Learning - Mitchell (Halaman 35)
- Deep Learning - Goodfellow (Halaman 17-20)

Konsep Dasar Latihan Backpropagation

Backpropagation adalah algoritma pembelajaran untuk Jaringan Saraf Tiruan (JST) yang bertujuan meminimalkan error dengan menyesuaikan bobot secara iteratif. Proses ini melibatkan dua tahap utama: **forward propagation** (menghitung output) dan **backward propagation** (menghitung error dan menyesuaikan bobot). Contoh penerapan dasar dapat dilihat pada jaringan dengan: - 2 unit input (x_1, x_2) - 1 unit hidden (h) - 3 unit output (o_1, o_2, o_3) - Fungsi aktivasi sigmoid: $\sigma(z) = 1/(1+e^{-z})$ **Alur perhitungan:** 1. Hitung output hidden layer: $h = \sigma(w_0 + w_1x_1 + w_2x_2)$ 2. Hitung output akhir: $o_k = \sigma(w_3 + w_4h)$ untuk setiap unit output